

Trekking Management Application — Safarnama

Project Report — Modern Application Development I

1. Author

Name: Maitri Jain

Roll Number: 23f2003837

Student Email: 23f2003837@ds.study.iitm.ac.in

Course: Modern Application Development I

Project: Trekking Management Application V1

GitHub Repository: <https://github.com/23f2003837/Trekking-Management-App-V1>

I am a BS Data Science student at IIT Madras with interests in web development, trekking, and esoteric cinema. This project reflects both my technical learning and personal wanderlust.

2. Description

Safarnama is a role-based Trekking Management Application built using Flask and SQLite. It allows Admin, Trek Staff, and Trekkers to interact with the system based on their roles — admins manage treks and users, staff manage assigned treks and participants, and trekkers browse, book, and track their adventures. The application was built with deep understanding of every component for a live viva examination.

3. AI / LLM Usage Declaration

AI/LLM Used: Claude (Anthropic) — approximately 20–25% usage overall.

Claude was used as a tutoring and code-review assistant. The specific uses were:

- Concept explanation — Asked Claude to explain Flask Blueprints, decorators, session management, and SQLAlchemy relationships. I read the explanations, understood them, and implemented the code myself.
- Code review and debugging — Shared code snippets to find bugs (e.g., IntegrityError in booking, missing table-responsive wrappers). I reviewed each suggestion, decided what to accept or reject, and wrote the final fix.
- Landing page generation — Gave Claude a detailed design brief (Hindi copywriting, teal/sage color palette, wanderlust theme) and it generated the HTML/CSS. I reviewed, adapted, and integrated it into the base template.
- Report and script formatting — Helped structure the report and format tables. All content was written and verified by me.

What I Wrote Myself:

- All Flask routes and business logic
- Database models and relationships (User, Trek, Booking, StaffProfile)
- Role-based decorators (login_as_admin, login_as_staff, login_as_trekker)
- Staff approval workflow and blacklist enforcement
- Booking, cancellation, and reactivation logic
- Trek creation, editing, and soft-delete logic
- All validation (date checks, slot limits, duplicate prevention)
- All templates except the landing page

My Confidence Level: I understand every line of code in this project. I can explain any route, modify any feature, and justify every design decision. Claude was a tutor and assistant — the code is mine.

4. Frameworks and Libraries Used

Framework / Library	Version	Purpose
Flask	3.1.3	Web framework, routing, session management, blueprint architecture
Flask-SQLAlchemy	3.1.1	ORM for SQLite database modelling and querying
SQLite	—	Lightweight relational database stored as a single file, created programmatically
Jinja2	—	Server-side HTML templating with template inheritance
Bootstrap	5.3	Responsive frontend CSS framework
Werkzeug	—	Password hashing using PBKDF2-SHA256
python-dotenv	—	Secure SECRET_KEY management via .env file
Google Fonts (Playfair Display)	—	Typography for landing page

5. ER Diagram

User (id PK, name, email UNIQUE NOT NULL, password NOT NULL, role [admin/staff/trekker] DEFAULT trekker, is_approved BOOLEAN DEFAULT False, is_blacklisted BOOLEAN DEFAULT False, profile_pic)
Reason: Single table for all roles allows unified authentication. Role column drives access control. is_approved gates staff login until admin approves. is_blacklisted enforces live access revocation on every request.

Trek (id PK, name, location, difficulty [easy/moderate/hard], duration AUTO-CALCULATED, available_slots, total_slots, start_date, end_date, status [pending/open/closed/ongoing/completed/cancelled], assigned_staff_id FK→User.id NULLABLE, description)
Reason: Status field encodes full trek lifecycle. Duration is auto-calculated from dates to prevent inconsistency. assigned_staff_id is nullable because treks may exist before staff assignment. Universal soft-delete: cancelled status replaces physical deletion to preserve booking history.

Booking (id PK, user_id FK→User.id NOT NULL, trek_id FK→Trek.id NOT NULL, booking_date DATETIME DEFAULT NOW, status [booked/cancelled/completed] DEFAULT booked, payment_status DEFAULT pending, UNIQUE CONSTRAINT on user_id+trek_id)
Reason: UniqueConstraint prevents duplicate bookings at database level as a safety net. Booking status transitions to completed when staff marks trek as completed, enabling trekker history tracking.

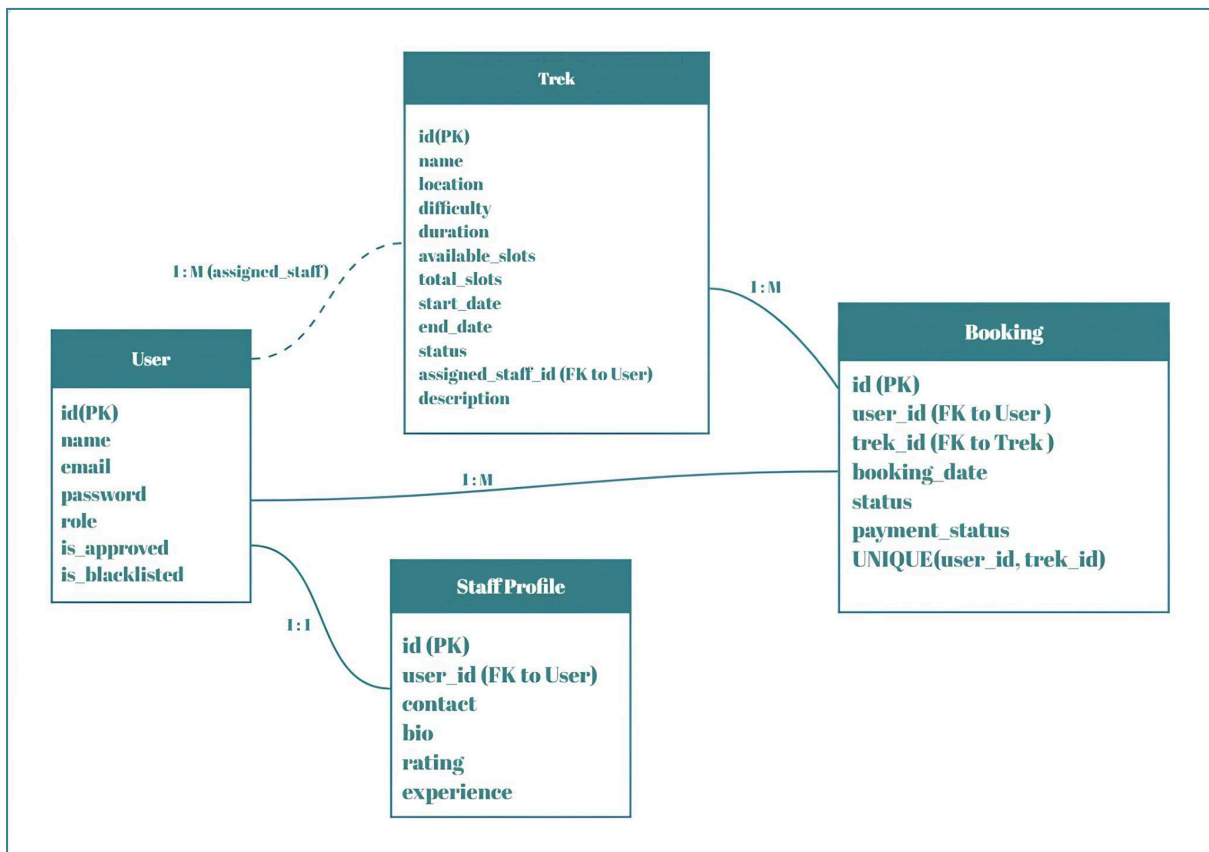
StaffProfile (id PK, user_id FK→User.id UNIQUE NOT NULL, contact, bio, rating FLOAT DEFAULT 0.0, experience FLOAT DEFAULT 0.0)
Reason: Separated from User to keep User table clean. One-to-one relationship enforced via UNIQUE on user_id.

The application uses four core tables: User, Trek, Booking, and StaffProfile.

- User (1) — (M) Booking: a user can make many bookings.

- User (1) — (1) StaffProfile: a staff user has exactly one staff profile.
- Trek (1) — (M) Booking: a trek can have many bookings.
- User (1) — (M) Trek, via assigned_staff_id: a staff user can be assigned to many treks, but each trek has at most one assigned guide (nullable — a trek may have no guide assigned yet).

ENTITY RELATIONSHIP DIAGRAM



6. API Resource Endpoints

A JSON API endpoint was implemented for trek data:

GET /api/treks — Returns all open treks as a JSON array. No authentication required, designed for potential third-party or mobile app consumption.

Response format:

```

JSON
{
  "treks": [
    {
      "id": 1,
      "name": "Kedarnath",
      "location": "Uttarakhand",
    }
  ]
}
  
```

```

        "difficulty": "hard",
        "available_slots": 8,
        "start_date": "2026-08-01",
        "end_date": "2026-08-06",
        "duration": 6,
        "description": "...",
    }
]
}

```

7. Architecture and Features

The project follows Flask's Application Factory Pattern. The app is initialized in `app/init.py` via `create_app()`, which configures the database, registers blueprints, and loads settings from `config.py`. Routes are organized in `app/routes.py` as a single Blueprint (main). Database models are defined in `app/models.py`. HTML templates use Jinja2 inheritance from a single `base.html`. Custom decorators in `app/decorators.py` (`login_as_admin`, `login_as_staff`, `login_as_trekker`) enforce role-based access on every protected route, with shared helper functions `check_login()` and `check_blacklisted()` eliminating code duplication. The admin is seeded programmatically via `seed.py`.

Core Features Implemented:

- Manual session-based authentication (no Flask-Login) with role-specific redirects
- Staff self-registration with admin approval flow (Pending/Approved/Blacklisted tabs)
- Full Admin CRUD for treks with auto-calculated duration, soft-delete preserving booking history
- Sequential trek status progression enforced at code level (closed→ongoing→completed only)
- Live blacklist enforcement — checked fresh from database on every protected request, not from stale session data
- Trekker booking with slot management, duplicate prevention (database UniqueConstraint + Python check), and re-booking after cancellation
- Booking cascade — when staff marks trek as completed, all associated booked statuses transition to completed automatically
- Three-parameter trek search (name, location, difficulty) for trekkers
- Staff trek ownership verification on every management action

Additional Features:

- Universal soft-delete for treks with cancelled archive page
- Trekker, staff, and admin booking/guiding history pages
- Staff profile management (contact, bio, experience, rating)
- JSON API endpoint at `/api/treks`
- Responsive UI using Bootstrap 5.3 with table-responsive wrappers
- Custom wanderlust-themed landing page (Safarnama) with original Hindi copywriting inspired by Indian travel cinema

8. Demo Video

Link to the Video - [Google Drive Link](#)